

Chapter8 内排序

1

请模拟快速排序的分割函数，给出下面序列的第一趟排序过程，要求写出每一次 交换后的序列状态（当前 pivot为末尾元素，从小到大排序） {70,89,63,39,87,92,22,87 ,12,6,70 ,94,17,31,11,50}

初始情况： pivot = 50
第 1 趟分割结果： Pivot 50 位于索引 7，序列被分为两部分：

- 左侧（小于或等于 pivot）： {39, 22, 12, 6, 17, 31, 11}
 - 右侧（大于 pivot）： {63, 70, 70, 94, 87, 92, 89, 87}
- 交换变化：
- {39, 89, 63, 70, 87, 92, 22, 87, 12, 6, 70, 94, 17, 31, 11, 50}
- {39, 22, 63, 70, 87, 92, 89, 87, 12, 6, 70, 94, 17, 31, 11, 50}
- {39, 22, 12, 70, 87, 92, 89, 87, 63, 6, 70, 94, 17, 31, 11, 50}
- {39, 22, 12, 6, 87, 92, 89, 87, 63, 70, 70, 94, 17, 31, 11, 50}
- {39, 22, 12, 6, 17, 92, 89, 87, 63, 70, 70, 94, 87, 31, 11, 50}
- {39, 22, 12, 6, 17, 31, 89, 87, 63, 70, 70, 94, 87, 92, 11, 50}
- {39, 22, 12, 6, 17, 31, 11, 87, 63, 70, 70, 94, 87, 92, 89, 50}
- {39, 22, 12, 6, 17, 31, 11, 50, 63, 70, 70, 94, 87, 92, 89, 87}

2

请分别利用高位优先法和低位优先法对下面的序列进行基数排序，并给出每一趟 分桶后的收集结果（高位优先法未收集，给出各桶中顺序的数据即可） {70,89,63,39,87,92,22,87 ,12,6,70 ,94,17,31,11,50}

低位优先法（LSD）：

1. 第一趟（按个位数排序）排序后收集结果： {70, 70, 31, 11, 92, 22, 63, 93, 94, 6, 87, 87, 89}
2. 第二趟{6, 11, 31, 22, 92, 63, 93, 70, 70, 87, 87, 89, 94}
3. 第三趟 {6, 11, 22, 31, 63, 70, 70, 87, 87, 89, 92, 93, 94}

最终序列为： {6, 11, 22, 31, 63, 70, 70, 87, 87, 89, 92, 93, 94}

高位优先法（MSD）

1. 第一趟{6, 11, 12, 22, 31, 39, 63, 70, 70, 87, 87, 89, 92, 94}
 2. {6, 11, 12, 22, 31, 39, 63, 70, 70, 87, 87, 89, 92, 94}
- 最终 {6, 11, 12, 22, 31, 39, 63, 70, 70, 87, 87, 89, 92, 94}

3

算法过程

算法步骤

- 生成子集和：利用二进制枚举法生成所有子集。

- 排序结果：将所有子集和放入列表中，排序输出。

```
// 计算子集和的函数
vector<int> subsetSums(const vector<int>& A) {
    int n = A.size();
    vector<int> subsetSums;

    for (int mask = 0; mask < (1 << n); ++mask) {
        int currentSum = 0;
        for (int i = 0; i < n; ++i) {
            if (mask & (1 << i)) {
                currentSum += A[i];
            }
        }

        subsetSums.push_back(currentSum);
    }

    // 排序
    sort(subsetSums.begin(), subsetSums.end());

    return subsetSums;
}
```

时间复杂度

排序和求和时间复杂度都是 $O(n \cdot 2^n)$ 所以总时间复杂度是 $O(n \cdot 2^n)$